



SAP HANA SQL Reference Guide for SAP HANA Platform

Generated on: 2026-10-03 19:09:38 GMT+0000

SAP HANA Platform | 2.0 SPS 08

Public

Original content: https://help.sap.com/docs/SAP_HANA_PLATFORM/4fe29514fd584807ac9f2a04f6754767?locale=en-US&state=PRODUCTION&version=2.0.08

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

SQL Reference

[Introduction to SQL](#)

This chapter describes the SAP HANA database implementation of Structured Query Language (SQL).

[SQL Notation Conventions](#)

SQL syntax notation conventions used in this guide.

[Data Types](#)

A data type defines the characteristics of a data value. A special value of NULL is included in every data type to indicate the absence of a value.

[Reserved Words](#)

Reserved words are words which have a special meaning to the SQL parser in the SAP HANA database and cannot be used as when defining an identifier.

[Operators](#)

Use operators to perform arithmetic operations in expressions. Operators can be used for calculation, value comparison, or to assign values.

[Expressions](#)

An expression is a clause that can be evaluated to return values.

[Predicates](#)

[Session Variables](#)

[SQL Functions](#)

Documents the built-in SQL functions that are provided with SAP HANA.

[SQL Statements](#)

SAP HANA supports many SQL statements to allow you to perform such tasks as create database objects, administer your system, and manipulate data.

[System Limitations](#)

Limitations to take into consideration when administering an SAP HANA database.

[SQL Error Codes](#)

Each SAP HANA error is identified by a numeric error code and has a short descriptive text.

[ANSI SQL Compliance](#)

Information about SAP HANA's compliance with the mandatory features of the ANSI SQL/2016 standard.

Introduction to SQL

This chapter describes the SAP HANA database implementation of Structured Query Language (SQL).

- [SQL](#)
- [Supported languages and code pages](#)
- [Comments](#)
- [Identifiers](#)
- [Identifiers and case sensitivity](#)
- [User names and passwords](#)
- [Quotation marks](#)

SQL

SQL stands for Structured Query Language. It is a standardized language for communicating with a relational database. SQL is used to retrieve, store or manipulate information in the database.

SQL statements perform the following tasks:

- Schema definition and manipulation
- Data manipulation
- System management
- Session management
- Transaction management

Supported languages and code pages

The SAP HANA database supports Unicode to allow the use of all languages in the Unicode Standard and 7 Bit ASCII code page without restriction.

Comments

You can add comments to improve the readability and maintainability of your SQL statements. Comments are delimited in SQL statements as follows:

- Double hyphens "--". Everything after the double hyphen until the end of a line is ignored by the SQL parser.
- "/*" and "*/". This style of commenting is used to place comments on multiple lines. All text between the opening "/*" and closing "*/" is ignored by the SQL parser.

Identifiers

Identifiers are used to represent names used in SQL statement including table name, view name, synonym name, column name, index name, function name, procedure name, user name, role name, and so on. There are two kinds of identifiers, undelimited identifiers and delimited identifiers.

- Undelimited table and column names must start with a letter and contain only letters, digits, or underscores "_".
- Delimited identifiers are enclosed in the delimiter, double quotes. The identifier can then contain any character including special characters. "AB\$%CD" is a valid identifier name for example.
- Limitations:
 - "_SYS_" is reserved exclusively for database engine and is therefore not allowed at the beginning of schema object names.
 - The role name and user name must be specified as undelimited identifiers.
 - The maximum length for identifiers is 127 characters.

Identifiers and case sensitivity

Identifiers without double-quotes in SQL syntax are converted to upper case when processed by the server. For example, the statement `CREATE COLUMN TABLE MyTAB . . .` creates a table called MYTAB, whereas `CREATE COLUMN TABLE "MyTab"` creates a table called MyTab--and both tables can co-exist in the database.

Specifying identifiers without double-quotes is allowed but can cause ambiguity later when querying or performing operations on objects where casing in the identifier name is significant. A recommendation is to standardize to using double-quotes around all identifiers in SQL statements where ambiguity may be a concern.

User names and passwords

User names may contain underscores and hyphens without requiring quotes. For a list of unpermitted characters in user names, see Unpermitted Characters in User Names topic in the *HANA Database Admin Guide*.

The following user name formats are unsupported:

- User names starting with double hyphens (for example, - -TESTUSER)
- User names containing only number signs (#) and hyphens (for example, #-##-)

Passwords must be a minimum length of 8. For more information on passwords see the Password Policy Configuration Options topic in the *HANA Database Admin Guide*

Quotation marks

Single quotation marks are used to delimit string literals. A single quotation mark itself can be represented using two single quotation marks. Double quotation marks are used to delimit identifiers. A double quotation mark itself can be represented using two double quotation marks.

Related Information

[M_INIFILE_CONTENTS System View](#)

[AUDIT_LOG System View](#)

[AUDIT_POLICIES System View](#)

[Unpermitted Characters in User Names](#)

[Password Policy Configuration Options](#)

[SQL Notation Conventions](#)

SQL Notation Conventions

SQL syntax notation conventions used in this guide.

This reference uses syntax notation very similar to BNF (Backus-Naur Form), a notation technique used to define programming languages.

Symbols

Symbol	Description
< >	Angle brackets are used to surround the name of a syntactic element (BNF nonterminal) of the SQL language.
::=	The definition operator is used to provide definitions of the element appearing on the left side of the operator in a production rule.
[]	Square brackets are used to indicate optional elements in a formula. Optional elements can be specified or omitted.

Symbol	Description
{ }	Braces group elements in a formula. Repetitive elements (zero or more elements) can be specified within brace symbols.
	The alternative operator indicates that the portion of the formula following the bar is an alternative to the portion preceding it.
[...]	Ellipsis with square brackets around it indicates optional repetition of the preceding element or grouped elements. For example, if you can specify one or more columns for an option and must separate them by commas, this is expressed as <code><column_name> [, ...]</code> . An example of grouped elements where a comma separator is not required looks like this: { <code><column_name> <data_type></code> } [...]
!!	Introduces normal English text. This is used when the definition of a syntactic element is not expressed in BNF.

Lowest Terms Representations

Throughout this manual, each syntax term is defined to one of the lowest term representations shown below.

`<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

`<letter> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v
| A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V`

`<any_character> ::= !!any character.`

`<comma> ::= ,`

`<dollar_sign> ::= $`

`<double_quotes> ::= "`

`<greater_than_sign> ::= >`

`<hash_symbol> ::= #`

`<left_bracket> ::= [`

`<left_curly_bracket> ::= {`

`<lower_than_sign> ::= <`

`<period> ::= .`

`<pipe_sign> ::= |`

`<right_bracket> ::= ]`

`<right_curly_bracket> ::= }`

`<sign> ::= + | -`

```

<single_quote> ::= '

<underscore> ::= _

<apostrophe> ::= <single_quote>

<approximate_numeric_literal> ::= <mantissa>E<exponent>

<cesu8_restricted_characters> ::= <double_quote> | <dollar_sign> | <single_quote> | <sign> | <period>

<exact_numeric_literal> ::= <unsigned_integer>[<period>[<unsigned_integer>]]
                           | <period><unsigned_integer>

<exponent> ::= <signed_integer>

<hostname> ::= {<letter> | <digit>}[{ <letter> | <digit> | <period> | - } [...] ]

<identifier> ::= <simple_identifier> | <special_identifier>

<mantissa> ::= <exact_numeric_literal>

<numeric_literal> ::= <signed_numeric_literal> | <signed_integer>

<password> ::= {
    <letter> [ { <letter_or_digit> | # | $ }[...] ]
  | <digit> [ <letter_or_digit> [...] ]
  | <special_identifier> }

<port_number> ::= <unsigned_integer>

<schema_name> ::= <unicode_name>

<simple_identifier> ::= {<letter> | <underscore>} [{<letter> | <digit> | <underscore> | <hash_symbol> } [...] ]

<special_identifier> ::= <double_quotes><any_character>...<double_quotes>

<signed_integer> ::= [<sign>] <unsigned_integer>

<signed_numeric_literal> ::= [<sign>] <unsigned_numeric_literal>

<string_literal> ::= <single_quote>[<any_character>...]<single_quote>

<unicode_name> ::= !! CESU-8 string excluding any characters listed in <cesu8_restricted_characters>

<unsigned_integer> ::= <digit>...

<unsigned_numeric_literal> ::= <exact_numeric_literal> | <approximate_numeric_literal>

<user_name> ::= <unicode_name>

```

Data Types

A data type defines the characteristics of a data value. A special value of NULL is included in every data type to indicate the absence of a value.

Classification of Data Types

In the SAP HANA database, each data type can be classified by its characteristics as follows:

Classification of data types

Classification	Data Type
Datetime types	DATE, TIME, SECONDDATE, TIMESTAMP
Numeric types	TINYINT, SMALLINT, INTEGER, BIGINT, SMALLDECIMAL, DECIMAL, REAL, DOUBLE
Boolean type	BOOLEAN
Character string types	VARCHAR, NVARCHAR, ALPHANUM, SHORTTEXT
Binary types	VARBINARY
Large Object types	BLOB, CLOB, NCLOB, TEXT
Multi-valued types	ARRAY
Spatial types	ST_GEOMETRY, ST_POINT

Typed Constants

A constant is a symbol that represents a specific fixed data value.

Character string constant

A character string constant is enclosed in single quotation marks, for example: 'Brian' or '100'.

Unicode strings have a similar format to character strings but are preceded by an N identifier (N stands for National Language in the SQL-92 standard).

```
SELECT 'Brian' "character string 1", '100' "character string 2", N'abc' "unicode string" FROM DUMMY;
```

The example above returns the following results:

character String 1	character String 2	unicode string
Brian	100	abc

Number constant

A number constant is represented by a string of numbers that are not enclosed in quotation marks. Numbers may contain a decimal point or a scientific notation. For example, 123, 123.4, or 1.234e2.

A hexadecimal number constant is a string of hexadecimal numbers and has the prefix 0x. For example, 0x0abc.

```
SELECT 123 "integer", 123.4 "decimal1", 1.234e2 "decimal2", 0x0abc "hexadecimal" FROM DUMMY;
```

The example above returns the following results:

integer	decimal1	decimal2	hexadecimal
123	123.4	123.4	2,748

Binary string constant

A binary string has the prefix X and is a string of hexadecimal numbers that are enclosed in quotation marks. For example, X'00abcd' or x'dcba00'.

```
SELECT X'00abcd' "binary string 1", x'dcba00' "binary string 2" FROM DUMMY;
```

The example above returns the following results:

binary string 1	binary string 2
00ABCD	DCBA00

Date/Time/Timestamp constant

Date, Time, and Timestamp each have the following prefixes:

- date'2010-01-01'
- time'11:00:00.001'
- timestamp'2011-12-31 23:59:59'

```
SELECT date'2010-01-01' "date", time'11:00:00.001' "time", timestamp'2011-12-31 23:59:59' "timestamp" FROM DUMMY;
```

The example above returns the following results:

date	time	timestamp
Jan 1, 2010	11:00:00 AM	Dec 31, 2011 11:59:59.0 PM

Binary Data Types

Binary types are used to store bytes of binary data.

A value of type binary can be converted to a value of type (N)VARCHAR if its size is smaller than or equal to 8192. It can therefore be used like a value of type (N)VARCHAR except for full text search operations and numeric operations.

VARBINARY

The VARBINARY(<n>) data type is used to store binary data of a specified maximum length in bytes, where <n> indicates the maximum length and is an integer between 1 and 5000. If the length is not specified, then the default is 1.

Boolean Data Type

The BOOLEAN data type stores boolean values, which are TRUE, FALSE, and UNKNOWN, where UNKNOWN is a synonym of NULL.

When the client does not support a boolean type, it returns 1 for TRUE and 0 for FALSE.

The following example returns TRUE or 1 for boolean:

```
CREATE ROW TABLE TEST (A BOOLEAN);
INSERT INTO TEST VALUES (TRUE);
INSERT INTO TEST VALUES (FALSE);
INSERT INTO TEST VALUES (UNKNOWN);
INSERT INTO TEST VALUES (NULL);

SELECT A "boolean" FROM TEST WHERE A = TRUE;
```

Although predicates and boolean expressions can both have the same values (TRUE, FALSE, UNKNOWN), they are not the same. Therefore, you cannot use boolean type comparisons to compare predicates, or use predicates where boolean expressions should be used.

For example, the following statement does not work:

```
SELECT * FROM DUMMY WHERE ( 'A'>'B' ) = ( 'C'>'D' );
```

The following statement is the correct way to achieve the results desired from the statement above:

```
SELECT * FROM DUMMY WHERE
CASE WHEN ( 'A'>'B' ) THEN TRUE WHEN NOT ( 'A'>'B' ) THEN FALSE ELSE NULL END=
CASE WHEN ( 'C'>'D' ) THEN TRUE WHEN NOT ( 'C'>'D' ) THEN FALSE ELSE NULL END;
```

Related Information

[TO_BOOLEAN Function \(Data Type Conversion\)](#)

Character String Data Types

Character string data types are used to store values that contain character strings. VARCHAR data types contain 7-bit ASCII character strings, and NVARCHAR are used for storing Unicode character strings.

Collation expressions are not supported, and values of string data type are compared using a binary comparison.

Character string data types in SAP HANA use 7-bit ASCII. Extended ASCII characters are converted into corresponding Unicode characters. If the data includes anything other than 7-bit ASCII characters, use Unicode character string types, such as NVARCHAR and NCLOB.

i Note

The SAP HANA database does not officially support the CHAR and NCHAR datatypes. Even though they are available for use, they are only for legacy support and consistent behavior is not guaranteed for values of these types between a row table and a column table. Use VARCHAR and NVARCHAR instead.

VARCHAR

The VARCHAR(<n>) data type specifies a variable-length character string, where <n> indicates the maximum length in bytes and is an integer between 1 and 5000. If the length is not specified in DDL statements, then the default of 1 is used.

If the VARCHAR(<n>) data type is used in a DML query, for example CAST (A as VARCHAR(n)), then <n> indicates the maximum length of the string in characters. If the length is not specified, then the default of 5000 is used. For non 7-bit ASCII character-based strings, use NVARCHAR.

NVARCHAR

The NVARCHAR(<n>) data type specifies a variable-length Unicode character set string, where <n> indicates the maximum length in characters and is an integer between 1 and 5000. If the length is not specified in DDL statements, then the default of 1 is used.

If the NVARCHAR(<n>) data type is used in a DML query, for example `CAST (A as NVARCHAR(n))`, then <n> indicates the maximum length of the string in characters. If the length is not specified, then the default of 5000 is used. For 7-bit ASCII character-based strings only, use VARCHAR.

ALPHANUM

The ALPHANUM(<n>) data type specifies a variable-length character string that contains alpha-numeric characters, where <n> indicates the maximum length and is an integer between 1 and 127.

Sorting among values of type ALPHANUM is performed in alpha-representation. In the case of a purely numeric value, this means that the value can be considered as an alpha value with leading zeros.

SHORTTEXT

The SHORTTEXT(<n>) data type specifies a variable-length character string that supports text search features and string search features. This data type can be defined for column tables but not for row tables. This is not a standalone SQL type. Selecting a SHORTTEXT(<n>) column yields a column of type NVARCHAR(<n>).

```
<shorttext_type> ::= SHORTTEXT ( <unsigned_integer> ) <elem_list_shorttext>
<elem_list_shorttext> ::= <fulltext_elem> [{, <fulltext_elem>}...]
```

Data Type Conversion

Both implicit and explicit data type conversions are allowed in the SAP HANA database.

i Note

SAP HANA does not support conversions between LOB data types.

Explicit type conversion

The data type of an expression result – for example, a field reference, a function on fields, or literals – can be converted using the following functions:

- CAST function
- TO_ALPHANUM function
- TO_BIGINT function
- TO_VARBINARY function
- TO_BLOB function
- TO_CLOB function
- TO_DATE function
- TO_DATS function
- TO_DECIMAL function
- TO_DOUBLE function
- TO_INTEGER function

- TO_INT function
- TO_NCLOB function
- TO_NVARCHAR function
- TO_REAL function
- TO_SECONDDATE function
- TO_SMALLINT function
- TO_TINYINT function
- TO_TIME function
- TO_TIMESTAMP function
- TO_VARCHAR function

Implicit type conversion

When a given set of operand/argument types does not match what an operator/function expects, a type conversion is carried out by the SAP HANA database. This conversion only occurs if a relevant conversion is available and if it makes the operation/function executable. For example, a comparison of BIGINT and VARCHAR is performed by implicitly converting VARCHAR to BIGINT. With the exception of TIME and TIMESTAMP data types, explicit conversions can be used for implicit conversions. TIME and TIMESTAMP data types can be converted reciprocally by using the TO_TIME(TIMESTAMP) and TO_TIMESTAMP(TIME) functions.

Implicit Type Conversion Examples

Input Expression	Transformed Expression with Implicit Conversion
BIGINT > VARCHAR	BIGINT > BIGINT(VARCHAR)
BIGINT > DECIMAL	DECIMAL(BIGINT) > DECIMAL
TIMESTAMP > DATE	TIMESTAMP > TIMESTAMP(DATE)
DATE > TIME	Returns an error because conversion cannot occur between DATE and TIME

The rules shown in the tables below are applicable to both implicit and explicit conversion except for TIME to TIMESTAMP conversion. Only explicit conversions are allowed for converting the TIME data type to the TIMESTAMP data type using the TO_TIMESTAMP or CAST functions.

In the tables below, **OK** means that data type conversions are allowed without any checks; **CHK** means that the data type can be converted if the data is valid for the target type; and - means that data type conversion is not allowed.

Numeric Data Type Conversion table

Target/Source	TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL	DECIMAL(<p>, <s>)	SMALLDECIMAL	REAL	DOUBLE
TINYINT	-	OK	OK	OK	OK	OK	OK	OK	OK
SMALLINT	CHK	-	OK	OK	OK	OK	OK	OK	OK
INTEGER	CHK	CHK	-	OK	OK	OK	OK	OK	OK
BIGINT	CHK	CHK	CHK	-	OK	CHK	CHK	CHK	OK
DECIMAL	CHK	CHK	CHK	CHK	-	CHK	CHK	CHK	OK

Target/Source	TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL	DECIMAL(<p>,<s>)	SMALLDECIMAL	REAL	DOUBLE
DECIMAL(<p>,<s>)	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK
SMALLDECIMAL	CHK	CHK	CHK	CHK	OK	CHK	-	CHK	CHK
REAL	CHK	CHK	CHK	CHK	OK	CHK	CHK	-	OK
DOUBLE	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK	-
VARCHAR	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK
NVARCHAR	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK

Datetime Data Type Conversion Table

Target/ Source	TIME	DATE	SECONDDATE	TIMESTAMP	VARCHAR	NVARCHAR
TIME	-	-	-	-	OK	OK
DATE	-	-	OK	OK	OK	OK
SECONDDATE	TIME	DATE	-	TIMESTAMP	OK	OK
TIMESTAMP	TIME	DATE	SECONDDATE	-	OK	OK
VARCHAR	CHK	CHK	CHK	CHK	-	OK
NVARCHAR	CHK	CHK	CHK	CHK	OK	-

Character String Data Type Conversion Table

Target/ Source	VARBINARY	VARCHAR	NVARCHAR	ALPHANUM
VARBINARY	-	-	-	-
ALPHANUM	-	OK	OK	-
VARCHAR	OK	-	OK	OK
NVARCHAR	OK	OK	-	OK

Data Type Precedence

Data type precedence specifies that the data type with the lower precedence is converted to the data type with the higher precedence.

The following list specifies the precedence of data types, with **TIMESTAMP** being the highest and **VARBINARY** being the lowest:

- **TIMESTAMP**
- **SECONDDATE**
- **DATE**
- **TIME**
- **DOUBLE**

- REAL
- DECIMAL
- SMALLDECIMAL
- DECIMAL(<precision>, <scale>)
- BIGINT
- INTEGER
- SMALLINT
- TINYINT
- NCLOB
- NVARCHAR
- CLOB
- VARCHAR
- BLOB
- VARBINARY

Spatial Data Type Conversion

Spatial data types can be converted using an explicit data type conversion function (TO_CHAR, TO_NCHAR, TO_VARCHAR, TO_NVARCHAR, TO_CLOB, TO_NCLOB, TO_BINARY, TO_VARBINARY). Implicit data conversion for spatial data types is not supported.

The following examples show how the data type conversion functions must be used to convert spatial data types:

```
Select TO_CHAR(NEW ST_Point('Point M( 1.0 2.0 NULL)')) from dummy;
Select TO_BINARY(NEW ST_Point('Point M( 1.0 2.0 NULL)')) from dummy;
Select TO_VARBINARY(NEW ST_Point('Point M( 1.0 2.0 NULL)')) from dummy;
-- and so on
```

Example

- When converting numeric types to DECIMAL or SMALLDECIMAL types, the least significant digits are rounded. For example, converting a BIGINT value such as 1234567890123456789 to SMALLDECIMAL becomes 1234567890123457000.
- When converting numeric types to any other numeric types, the least significant digits are truncated toward 0. The most significant digits cannot be truncated. Overflow errors occur if the converted value is incompatible with the target format. For example, converting a DECIMAL(10,2) value such as 12345.67 to BIGINT becomes 12345.
- When converting character string types to numeric types, no truncation is allowed, and an overflow error or invalid number error can be returned. For example, converting a VARCHAR value such as "12345.67" to BIGINT fails and returns an invalid number error. And converting a the VARCHAR value "256" to TINYINT generates and overflow error.
- When converting datetime data types to other datetime data types, the results are truncated to the target type when the source type is larger; otherwise, the default value is added. For example, you can converting the TIMESTAMP 2013-07-03 01:23:45.123456789 to a DATE data type returns 2013-07-03. Converting the DATE 2013-07-03 to a TIMESTAMP returns 2013-07-03 00:00:00.000000000. Converting the TIMESTAMP 2013-07-03 23:59:59.7777777 to a SECONDDATE returns 2013-07-03 23:59:59.

- When converting character string types to datetime types, the results are truncated to the target type, and an invalid DATE, TIME or TIMESTAMP error can be returned. For example, converting the VARCHAR value "2013-07-03 00:00:00.000000000" to a DATE returns 2013-07-03.

Datetime Data Types

Datetime data types are used to store date and time information.

DATE

The DATE data type consists of year, month, and day information to represent a date value. The default format for the DATE data type is YYYY-MM-DD. YYYY represents the year, MM represents the month, and DD represents the day. The range of the date value is between 0001-01-01 and 9999-12-31.

DATE is a synonym for DAYDATE.

TIME

The TIME data type consists of hour, minute, and second information to represent a time value. The default format for the TIME data type is HH24:MI:SS. HH24 represents the hour from 0 to 24, MI represents the minute from 0 to 59, and SS represents the second from 0 to 59.

SECONDDATE

The SECONDDATE data type consists of year, month, day, hour, minute and second information to represent a date with a time value. The default format for the SECONDDATE data type is YYYY-MM-DD HH24:MI:SS. YYYY represents the year, MM represents the month, DD represents the day, HH24 represents hours, MI represents minutes, and SS represents seconds. The range of the date value is between 0001-01-01 00:00:01 and 9999-12-31 24:00:00.

TIMESTAMP

The TIMESTAMP data type consists of date and time information. Its default format is YYYY-MM-DD HH24:MI:SS.FF7. FF <n> represents the fractional seconds where <n> indicates the number of digits in fractional part. The range of the time stamp value is between 0001-01-01 00:00:00.0000000 and 9999-12-31 23:59:59.9999999. LONGDATE is a synonym of TIMESTAMP.

The EMPTY value refers to the lowest possible value of each datetime type and ensures compatibility with ABAP.

Datetime Session Variables

The following datetime session variables can be used to override the system default datetime formats for the session. Use the SET statements to set session variables.

DATE_FORMAT

Specifies the default DATE format to apply for the session.

TIME_FORMAT

Specifies the default TIME format to apply for the session.

SECONDDATE_FORMAT

Specifies the default SECONDDATE format to apply for the session.

TIMESTAMP_FORMAT

Specifies the default TIMESTAMP format to apply for the session.

When setting the TIMESTAMP_FORMAT session variables, if you do not want to lose decimal point seconds when converting values, then specify SS.FF7 as the format for seconds instead of SS.

Example of how setting the session variable affects a default format::

```

CREATE TABLE T1(A TIMESTAMP);
INSERT INTO T1 VALUES ('2018/01/02 10:00:00'); --> OK
INSERT INTO T1 VALUES ('02/01/2018 10:00:00'); --> ERROR
SELECT TO_VARCHAR(A) FROM T1; --> 2018-01-02 10:00:00.0000000

SET 'TIMESTAMP_FORMAT' = 'DD/MM/YYYY HH:MI:SS';

CREATE TABLE T2(A TIMESTAMP);
INSERT INTO T2 VALUES ('2018/01/02 10:00:00'); --> ERROR
INSERT INTO T2 VALUES ('02/01/2018 10:00:00'); --> OK

SELECT TO_VARCHAR(A) FROM T2; --> 02/01/2018 10:00:00

```

Date Formats

The following datetime formats can be used when parsing a string into a DATETIME type and converting a DATETIME type value into a string value. The format for TIMESTAMP is the combination of DATE and TIME with additional support for fractional seconds. An empty date (0000-00-00) is a special value in SAP HANA. Even though an empty date looks like a NULL or unknown value, it is not. For example, the empty date can be represented as ' ', which behaves like an empty string. It also satisfies a IS NOT NULL predicate. Use the following statements to test the behavior of the empty date value:

```

CREATE ROW TABLE T (A INT, B DATE, C DATE);
INSERT INTO T VALUES (1, ' ', '0001-01-01');
INSERT INTO T VALUES (2, '0000-00-00', '0001-01-01');
INSERT INTO T VALUES (3, '0000-00-00', '0001-01-01');
INSERT INTO T VALUES (4, '0001-01-01', '0001-01-01');
INSERT INTO T VALUES (5, NULL, '0001-01-01');

SELECT * FROM T WHERE B = '00000000';
SELECT * FROM T WHERE B = ' ';
SELECT * FROM T WHERE B IS NOT NULL;
SELECT * FROM T;
SELECT DAYS_BETWEEN(B, C) FROM T;

```

Supported Formats for DATE

Format	Description	Examples
YYYY-MM-DD	Default format.	INSERT INTO my_tbl VALUES ('1957-06-13');
YYYY/MM/DD YYYY/MM-DD YYYY-MM/DD	YYYY is from 0001 to 9999, MM is from 1 to 12, and DD is from 1 to 31. If year has less than four digits, month has less than two digits, or day has less than two digits, then values are padded by one or more zeros. For example, a two-digit year like 45 is saved as year 0045, while a one digit	INSERT INTO my_tbl VALUES ('1957-06-13'); INSERT INTO my_tbl VALUES ('1957/06/13'); INSERT INTO my_tbl VALUES ('1957/06-13'); INSERT INTO my_tbl VALUES ('1957-06/13');

Format	Description	Examples
	month like 9 is saved as 09, and a one digit day like 2 is saved as 02.	
YYYYMMDD	The ABAP Data Type, DATES format.	INSERT INTO my_tbl VALUES ('19570613');
MON	The abbreviated name of month. For example, JAN. or DEC..	INSERT INTO my_tbl VALUES (TO_DATE('2040-Jan-10', 'YYYY-MON-DD')); INSERT INTO my_tbl VALUES (TO_DATE('Jan-10', 'MON-DD'));
MONTH	The name of month. For example, JANUARY - DECEMBER.	INSERT INTO my_tbl VALUES (TO_DATE('2040-January-10', 'YYYY-MONTH-DD')); INSERT INTO my_tbl VALUES (TO_DATE('January-10', 'MONTH-DD'));
RM	The Roman numeral month (I-XII; JAN = I).	INSERT INTO my_tbl VALUES (TO_DATE('2040-I-10', 'YYYY-RM-DD')); INSERT INTO my_tbl VALUES (TO_DATE('I-10', 'RM-DD'));
DDD	The day of the year (1-366).	INSERT INTO my_tbl VALUES (TO_DATE('204', 'DDD')); INSERT INTO my_tbl VALUES (TO_DATE('2001-204', 'YYYY-DDD'));

Time Formats

Supported formats for TIME

Format	Description	Examples
HH24:MI:SS	The default format.	
HH:MI[:SS][AM PM] HH12:MI[:SS][AM PM] HH24:MI[:SS]	HH is from 0 to 23. MI is from 0 to 59. SS is from 0 to 59. If a one digit hour, minute, or second is specified, then 0 is also inserted into the value. For example, 9:9:9 is saved as 09:09:09. HH12 indicates a 12 hour clock. HH24 indicates a 24 hour clock. Specify AM or PM as a suffix to indicate that	INSERT INTO my_tbl VALUES ('23:59:59'); INSERT INTO my_tbl VALUES ('3:47:39 AM'); INSERT INTO my_tbl VALUES ('9:9:9 AM'); INSERT INTO my_tbl VALUES (TO_TIME('11:59:59', 'HH12:MI:SS'));

Format	Description	Examples
	the time value is before or after midday.	
SSSSS	The seconds past midnight (0-86400).	INSERT INTO my_tbl VALUES (TO_TIME('12345', 'SSSSS'));

Timestamp Formats

Supported formats for TIMESTAMP

Format	Description	Examples
YYYY-MM-DD HH24:MI:SS.FF7	The default format.	
FF [1..7]	Fractional seconds have the range of 1 to 7 after the FF parameter to specify the number of digits in the fractional second portion of the datetime value returned. If a digit is not specified, then an error is returned.	INSERT INTO my_tbl VALUES (TO_TIMESTAMP('2011-05-11 12:59:59.999', 'YYYY-MM-DD H

Additional Formats for DATETIME

The following additional formats for DATETIME are supported.

While all of the formats below can be converted to strings (TO_VARCHAR function), conversion from string into D, DAY, DY, Q, W, and WW formats (for example, using TO_DATE and TO_TIMESTAMP functions) is not supported.

Format	Description
D	The day of the week (1-7).
DAY	The name of the day (MONDAY - SUNDAY).
DY	The abbreviated name of the day (MON - SUN).

Format	Description
MON	The abbreviated month name (JAN - DEC).
MONTH	The full month name (JANUARY - DECEMBER).
RM	The Roman numeral month (I - XII; I is for January).
Q	The quarter of the year (1, 2, 3, 4).
W	The week of the month (1-6).
WW	The week of the year (1-54).

Related Information

[SET \[SESSION\] Statement \(Session Management\)](#)

[M_SESSION_CONTEXT System View](#)

[Session Variables](#)

Large Object (LOB) Data Types

LOB (large objects) data types, such as CLOB, NCLOB, and BLOB, are used to store a large amount of data, such as text documents and images.

BLOB

The BLOB data type is used to store large amounts of binary data. BLOB values can be converted to VARBINARY.

CLOB

The CLOB data type is used to store large amounts of 7-bit ASCII character data. CLOB values can be converted to VARCHAR.

NCLOB

The NCLOB data type is used to store a large Unicode character object. NCLOB values can be converted to NVARCHAR.

TEXT

The TEXT data type enables text search features. This data type can be defined for column tables, but not for row tables. This is not a standalone SQL type. Selecting a TEXT column yields a column of type NCLOB.

A value of type TEXT cannot be converted implicitly to a value of type (N)VARCHAR, and string functions (UPPER, LOWER, and so on) cannot be applied directly to a value of type TEXT. Explicit conversion from a value of type TEXT to a value of type (N)VARCHAR is allowed. String functions can therefore be applied to the converted value.

For columns of type TEXT, the LIKE predicate is not supported.

```
<text_type> ::= TEXT <opt_fulltext_elem_list_text>
<opt_fulltext_elem_list_text> ::= <fulltext_elem> [{, <fulltext_elem>}...]
```

BINTEXT

The BINTEXT data type is similar to the TEXT data type and supports text search features, but it is possible to insert binary data. This data type can be defined for column tables, but not for row tables. This is not a standalone SQL type. Selecting a BINTEXT column yields a column of type NCLOB.

For values of type BINTEXT, the same restrictions apply as for values of type TEXT.

```

<bintext_type> ::= BINTEXT <opt_fulltext_elem_list_bintext>

<opt_fulltext_elem_list_bintext> ::= <fulltext_elem> [{, <fulltext_elem>}...]

```

The following syntax rules are common to TEXT, BINTEXT, and SHORTTEXT data types:

```

<fulltext_elem> ::= LANGUAGE COLUMN <column_name>
                  | LANGUAGE DETECTION ( <string_literal_list> )
                  | MIME TYPE COLUMN <column_name>
                  | <change_tracking_elem>
                  | FUZZY SEARCH INDEX <on_off>
                  | PHRASE INDEX RATIO <index_ratio>
                  | CONFIGURATION <string_literal>
                  | SEARCH ONLY <on_off>
                  | FAST PREPROCESS <on_off>
                  | TEXT ANALYSIS <on_off>
                  | MIME TYPE <string_literal>
                  | TOKEN SEPARATORS <string_literal>

<change_tracking_elem> ::= SYNC[HRONOUS]
                        | ASYNC[HRONOUS] [FLUSH [QUEUE] <flush_queue_elem>]

<flush_queue_elem> ::= EVERY <integer_literal> MINUTES
                    | AFTER <integer_literal> DOCUMENTS
                    | EVERY <integer_literal> MINUTES OR AFTER <integer_literal> DOCUMENTS

```

LOB types support the following operations:

- The LENGTH() function for values of type CLOB/NCLOB/BLOB, which returns the LOB length in bytes.
- The SUBSTR() function for values of type CLOB/NCLOB, which returns the substring of a (N)CLOB value.
- The COALESCE() function.
- The LIKE and CONTAINS predicate for values of type CLOB/NCLOB.
- The IS NULL predicate for values of type CLOB/NCLOB/BLOB.

LOB data types have the following restrictions:

- LOB columns cannot appear in ORDER BY or GROUP BY clauses.
- LOB columns cannot appear in FROM clauses as join predicates.
- LOB columns cannot appear in WHERE clauses as predicates other than LIKE (meaning that no comparison is allowed).
- LOB columns cannot appear in SELECT clauses as aggregate function arguments.
- LOB columns cannot appear in SELECT DISTINCT clauses.
- LOB columns cannot be used in set operations such as EXCEPT. UNION ALL is an exception.
- LOB columns cannot be used as primary keys.
- LOB columns cannot be used in CREATE INDEX statements.
- LOB columns cannot be used in statistics update statements.

Related Information

[System Limitations](#)

Multi-valued Data Types

Multi-valued types are used to store collections of values sharing the same data type.

For multi-valued data types (nested data types), SAP HANA supports all basic data types as element data types such as INTEGER, FLOAT, DECIMAL, VARCHAR, and so on. However, it does not support complex or unusual types like TEXT, LOB, and spatial types. SAP HANA also does not support multi-values of multi-values.

ARRAY

The ARRAY type is used to store collections of values sharing the same data type where each element is associated with exactly one ordinal position. Arrays can contain NULL values as elements to indicate the absence of a value. Arrays are immutable: adding, removing, or changing elements is not possible. Arrays with the WITHOUT DUPLICATES constraint cannot have the same non-NULL value more than once.

Supported functions and expressions for the ARRAY type

- `<ARRAY> || <ARRAY>` (concatenation) (SQLScript)
- `<ARRAY>[<index>]` (element access) (SQLScript)
- CARDINALITY Function
- MEMBER_AT Function
- [NOT] MEMBER OF Function
- SUBARRAY Function
- TRIM_ARRAY Function
- UNNEST (SQLScript)

Related Information

[Array Variables](#)

[UNNEST Function](#)

Numeric Data Types

Numeric data types are used to store numeric information.

Each numeric type below has a maximum value and minimum value. A numeric overflow exception is thrown if a value is smaller than the minimum value or greater than the maximum value. In order to comply with IEEE754, NaN and infinity are not supported. -0.0 is stored as +0.0.

Floating-point data types are stored in the system using binary numbers. The fractional part of these numbers is represented using a combination of 1/2, 1/4, 1/8, 1/16, and so on. For this reason, they cannot completely represent rational numbers with fractional digits. For example, 0.1 cannot be represented exactly by combining these binary fractions. In this case, you obtain inaccurate results when using a floating-point data type. This is the correct behavior for these data types. The following example demonstrates the behavior and returns 4.6999999999 for DOUBLE_SUM:

```
SELECT TO_DOUBLE(0.1) + TO_DOUBLE(4.6) AS DOUBLE_SUM FROM DUMMY;
```

For a list of numeric functions, see [Numeric Functions](#).

TINYINT

The TINYINT data type stores an 8-bit unsigned integer. The minimum value is 0 and the maximum value is 255.

SMALLINT

The SMALLINT data type stores a 16-bit signed integer. The minimum value is -32,768 and the maximum value is 32,767.

INTEGER

The INTEGER data type stores a 32-bit signed integer. The minimum value is -2,147,483,648 and the maximum value is 2,147,483,647.

BIGINT

The BIGINT data type stores a 64-bit signed integer. The minimum value is -9,223,372,036,854,775,808 and the maximum value is 9,223,372,036,854,775,807.

DECIMAL (<precision>, <scale>) or DEC (<p>, <s>)

DECIMAL(<p>, <s>) is the SQL standard notation for fixed-point decimals. <p> specifies precision, or the number of total digits (the sum of whole digits and fractional digits). <s> denotes scale, or the number of fractional digits. If a column is defined as DECIMAL(5, 4) for example, the numbers 3.14, 3.1415, 3.141592 are stored in the column as 3.1400, 3.1415, 3.1415, retaining the specified precision(5) and scale(4).

DECIMAL

Precision can range from 1 to 38. The scale can range from 0 to <p>. If the scale is not specified, then it defaults to 0.

If precision and scale are not specified, then DECIMAL becomes a floating-point decimal number. In this case, precision and scale can vary within the range of 1 to 34 for precision and -6,111 to 6,176 for scale, depending on the stored value.

For example, 0.0000001234 (1234E-10) has precision 4 and scale 10. 1.0000001234 (10000001234E-10) has precision 11 and scale 10. The value 1234000000 (1234E6) has precision 4 and scale 6.

SMALLDECIMAL

The SMALLDECIMAL data type is a floating-point decimal number. The precision and scale can vary within the range 1-16 for precision and -369-368 for scale, depending on the stored value. SMALLDECIMAL is supported on row and column store.

REAL

The REAL data type specifies a single-precision, 32-bit floating-point number.

DOUBLE

The DOUBLE data type specifies a double-precision, 64-bit floating-point number. The minimum value is -1.7976931348623157E308 and the maximum value is 1.7976931348623157E308. The smallest positive DOUBLE value is 2.2250738585072014E-308 and the largest negative DOUBLE value is -2.2250738585072014E-308.

FLOAT(<n>)

The FLOAT(<n>) data type specifies a 32-bit or 64-bit real number, where <n> specifies the number of significant bits and can range between 1 and 53.

If you use the FLOAT(<n>) data type, and <n> is smaller than 25, then the 32-bit REAL data type is used instead. If <n> is greater than or equal to 25, or if <n> is not declared, then the 64-bit DOUBLE data type is used.

Spatial Data Types

Spatial data types are used to store values that contain spatial data, such as points, lines, or polygons.

The following column types are supported in column tables only: ST_Point, ST_Geometry.

The column type ST_Point supports only the two-dimensional spatial data type ST_Point.

The column type ST_Geometry supports the following spatial data types:

- ST_CircularString
- ST_GeometryCollection
- ST_LineString
- ST_MultiLineString
- ST_MultiPoint
- ST_MultiPolygon
- ST_Point
- ST_Polygon

For more information on spatial data types, refer to the [SAP HANA Spatial Reference](#).

Reserved Words

Reserved words are words which have a special meaning to the SQL parser in the SAP HANA database and cannot be used as when defining an identifier.

The following list provides you with the current reserved words for the SAP HANA database. You can obtain this list by querying the RESERVED_KEYWORDS system view:

```
SELECT * FROM RESERVED_KEYWORDS ORDER BY RESERVED_KEYWORD;
```

In addition to the keywords listed below, avoid using the reserved keywords from the most recent ANSI SQL standard to ensure the compatibility of your SQL statements with subsequent SAP HANA database developments. However, if you do use any of them, we recommend placing them in uppercase and delimiting them with double quotation marks to ensure compatibility.

```
ALL
ALTER
AS
BEFORE
BEGIN
BOTH
CASE
CHAR
CONDITION
CONNECT
CROSS
CUBE
CURRENT_CONNECTION
CURRENT_DATE
CURRENT_SCHEMA
CURRENT_TIME
CURRENT_TIMESTAMP
CURRENT_TRANSACTION_ISOLATION_LEVEL
CURRENT_USER
CURRENT_UTCDATE
CURRENT_UTCTIME
CURRENT_UTCTIMESTAMP
CURRVAL
CURSOR
DECLARE
DISTINCT
```

ELSE
ELSEIF
END
EXCEPT
EXCEPTION
EXEC
FALSE
FOR
FROM
FULL
GROUP
HAVING
IF
IN
INNER
INOUT
INTERSECT
INTO
IS
JOIN
LATERAL
LEADING
LEFT
LIMIT
LOOP
MINUS
NATURAL
NCHAR
NEXTVAL
NULL
ON
ORDER
OUT
PRIOR
RETURN
RETURNS
REVERSE
RIGHT
ROLLUP
ROWID
SELECT
SESSION_USER
SET
SQL
START
SYSUOID
TABLESAMPLE
TOP
TRAILING
TRUE
UNION
UNKNOWN
USING
UTCTIMESTAMP
VALUES
WHEN
WHERE
WHILE
WITH

Operators

Use operators to perform arithmetic operations in expressions. Operators can be used for calculation, value comparison, or to assign values.

- [Unary and Binary Operators](#)
- [Operator Precedence](#)

- [Arithmetic Operators](#)
- [String Operators](#)
- [Comparison Operators](#)
- [Logical Operators](#)
- [Set Operators](#)

Unary and Binary Operators

Unary and binary operators

Operator	Operation	Format	Description
Unary	A unary operator applies to one operand or a single value expression.	operator operand	unary plus operator(+) unary negation operator(-) logical negation(NOT)
Binary	Binary A binary operator applies to two operands or two value expressions.	operand1 operator operand2	multiplicative operators (*, /) additive operators (+, -) comparison operators (=, !=, <, >, <=, >=) logical operators (AND, OR)

Operator Precedence

An expression can use multiple operators. If the number of operators is greater than one, the SAP HANA database evaluates them in order of operator precedence. You can use parentheses to change the order of evaluation, as expressions contained within parentheses are always evaluated first.

If parentheses are not used, the precedence of the various operators is as indicated by the table below. The SAP HANA database evaluates operators with equal precedence from left to right within an expression.

SQL operator precedence

Precedence	Operator	Operation
Highest	()	parentheses
	+, -	unary positive and negative operation
	*, /	multiplication, division
	+, -	addition, subtraction
		concatenation
	=, !=, <, >, <=, >=, IS NULL, LIKE, BETWEEN	comparison
	NOT	logical negation
	AND	conjunction

Precedence	Operator	Operation
Lowest	OR	disjunction

Arithmetic Operators

You use arithmetic operators to perform mathematical operations, such as adding, subtracting, multiplying, dividing and negation of numeric values.

Arithmetic operators

Operator	Description
-<expression>	Negation. If the expression is NULL, the result is NULL.
<expression> + <expression>	Addition. If either expression is NULL, the result is NULL.
<expression> - <expression>	Subtraction. If either expression is NULL, the result is NULL.
<expression> * <expression>	Multiplication. If either expression is NULL, the result is NULL.
<expression> / <expression>	Division. If either expression is NULL, or if the second expression is 0, an error is returned.

When the result precision is larger than 38 (the largest precision that the DECIMAL(p, s) type can hold), the result type of arithmetic operators is DECIMAL (floating-point decimal number) type.

The Result Precision and Scale of Arithmetic Operators on DECIMAL(p, s) Types Where e1 is DECIMAL(p1, s2) and e2 is DECIMAL(p2, s2)

Operator	Result Precision	Result Scale
e1 + e2	max(s1, s2) + max(p1-s1, p2-s2) + 1	max(s1, s2)
e1 - e2	max(s1, s2) + max(p1-s1, p2-s2) + 1	max(s1, s2)
e1 * e2	p1 + p2	s1 + s2
e1 / e2	p1 - s1 + s2 + max(6, s1 + p2 + 1)	max(6, s1 + p2 + 1)

String Operators

A concatenation operator combines two items, such as strings, expressions, or constants, into one.

Concatenation operators

Operator	Description
<expression> <expression>	String concatenation (two vertical bars). If either string is NULL, it returns NULL.

For VARCHAR or NVARCHAR type strings, leading or trailing spaces are retained. If either string is of data type NVARCHAR, the result has data type NVARCHAR and is limited to 64 MB in length. The maximum length for VARCHAR concatenation is also limited to 5000 characters.

Comparison Operators

<comparison_operation> ::= <expression1> <comparison_operator> <expression2>

Comparison operators

Operator	Description	Example
=	Equal to	SELECT * FROM students WHERE id = 25;
>	Greater than	SELECT * FROM students WHERE id > 25;
<	Less than	SELECT * FROM students WHERE id < 25;
>=	Greater than or equal to	SELECT * FROM students WHERE id >= 25;
<=	Less than or equal to	SELECT * FROM students WHERE id <= 25;
!=, <>	Not equal	SELECT * FROM students WHERE id != 25; SELECT * FROM students WHERE id <> 25;

Logical Operators

Search conditions can be combined using AND or OR operators. You can also negate them using the NOT operator.

Logical operators

Operator	Syntax	Description
AND	WHERE condition1 AND condition2	When using AND, the combined condition is TRUE if both conditions are TRUE, FALSE if either condition is FALSE, and UNKNOWN otherwise.
OR	WHERE condition1 OR condition2	When using OR, the combined condition is TRUE if either condition is TRUE, FALSE if both conditions are FALSE, and UNKNOWN otherwise.
NOT	WHERE NOT condition	The NOT operator is placed before a condition to negate the condition. The NOT condition is TRUE if condition is FALSE, FALSE if condition is TRUE, and UNKNOWN if condition is UNKNOWN.

Set Operators

Set operators allow multiple queries to be combined to return a single result set.

Operator	Returned Value
UNION	Combines the results of two or more select statements or query expressions
UNION ALL	Combines the results of two or more select statements or query expressions, including all duplicate rows.

Operator	Returned Value
INTERSECT	Combines the results of two or more select statements or query expressions, and returns all common rows.
EXCEPT	Takes the output from the first query and then removes the rows selected by the second query. MINUS is an accepted synonym for EXCEPT.

Related Information

[SELECT Statement \(Data Manipulation\)](#)

Expressions

An expression is a clause that can be evaluated to return values.

- [Case Expressions](#)
- [Function Expressions](#)
- [Aggregate Expressions](#)
- [JSON Object Expressions](#)
- [Subqueries in Expressions](#)

Syntax

```
<expression> ::= <case_expression>
                | <function_expression>
                | <aggregate_expression>
                | (<expression> )
                | ( <subquery> )
                | - <expression>
                | <expression> <operator> <expression>
                | <variable_name>
                | <constant>
                | [<correlation_name>.]<column_name>
```

Case Expressions

A case expression allows the user to use IF - THEN - ELSE logic without using SQLScript in SQL statements.

Syntax

```
<case_expression> ::= <simple_case_expression> | <search_case_expression>

<simple_case_expression> ::=
    CASE <expression>
    WHEN <expression> THEN <expression>
    [{ WHEN <expression> THEN <expression>}...]
    [ ELSE <expression>]
    END

<search_case_expression> > ::=
    CASE
    WHEN <condition> THEN <expression>
    [{ WHEN <condition> THEN <expression>}...]
```

```
[ ELSE <expression>]
END
```

```
<condition> ::= <condition> OR <condition> | <condition> AND <condition>
              | NOT <condition> | ( <condition> ) | <predicate>
```

If the expression following the CASE statement is equal to the expression following the WHEN statement, then the expression following the THEN statement is returned. Otherwise, the expression following the ELSE statement is returned if it exists.

While some short-circuiting behavior to improve performance may take place, when and whether all WHEN clauses are evaluated in a CASE expression is not guaranteed.

If all *<expression>* (this includes predicate expressions and/or scalar expressions) do not return an error, then the result of CASE expression is deterministic. That is, the CASE expression will always return a value (returning behavior) and will always return a value from the first matching part (return value behavior).

If any *<expression>* returns an error, then the result of CASE is not deterministic; it may return a value or throw an error depending on an evaluation order of expressions determined by the HANA Optimizer and execution engine. If in this scenario CASE does return a value, then the value will be from the first matching part.

Consider the following example:

```
CASE WHEN p1 THEN e1 WHEN p2 THEN e2 ELSE e3 END
```

In SAP HANA, p1, e1, p2, e2 and e3 can be evaluated in any order. But let's assume for example's sake that p1 would always return TRUE, and e2 would always throw an exception.

If the order chosen is p1 > e1 > p2 > e2 > e3, and since p1 returns TRUE, then e1 is evaluated and returned as the result of the CASE expression. The remainder of the expressions p2, e2, and e3, are never evaluated so an exception is not thrown for e2.

If the order chosen is : e1 > e2 > e3 > p1 > p2, and since e2 returns an exception, then the rest of e3, p1, p2 is never evaluated and an exception is thrown.

Either of these orders and outcomes are possible in SAP HANA, and this is true for both *<simple_case_expression>* and *<search_case_expression>*.

Function Expressions

SQL built-in functions can be used as expressions.

Syntax

```
<function_expression> ::= <function_name> ( <expression> [{, <expression>}...] )
```

Aggregate Expressions

An aggregate expression uses an aggregate function to calculate a single value from the values of multiple rows in one or more columns.

Syntax

```
<aggregate_expression> ::=
COUNT(*)
| COUNT ( DISTINCT <expression_list> )
| <agg_name> ( [ ALL | DISTINCT ] <expression> )
| STRING_AGG ( <expression> [, <delimiter>] [<aggregate_order_by_clause>])
```

`<agg_name> ::= CORR | CORR_SPEARMAN | COUNT | MIN | MEDIAN | MAX | SUM | AVG | STDDEV | VAR | STDI`
`<delimiter> ::= <string_constant>`
`<aggregate_order_by_clause> ::= ORDER BY <expression> [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]`

You can specify to sort the aggregate using the `<aggregate_order_by_clause>`. ASC sorts records in ascending order. DESC sorts records in descending order. By default, for ascending ordering, NULL values are returned first, and for descending they are returned last. You can override this behavior by using NULLS FIRST or NULLS LAST to explicitly specify NULL value ordering.

Aggregate expressions with DISTINCT: When you specify the DISTINCT keyword in a query, duplicate values are eliminated before calculations take place. The following aggregate functions are not supported for use with DISTINCT: STDDEV_POP, STDDEV_SAMP, VAR_POP, VAR_SAMP.

Aggregate name	Description
CORR	Computes the Pearson product momentum correlation coefficient between two columns. See the CORR function for more details.
CORR_SPEARMAN	Returns the Spearman's rank correlation coefficient of the values found in the corresponding rows of two columns. See the CORR_SPEARMAN function for more details.
COUNT	Counts the number of rows returned by a query. COUNT(*) returns the number of rows, regardless of the value of those rows and including duplicate values. COUNT(<expression>) returns the number of non-NULL values for that expression returned by the query. COUNT(DISTINCT <expression_list>) returns the number of distinct values for that expressions returned by the query, excluding rows with all NULL values for that expression.
MIN	Returns the minimum value of the expression.
MEDIAN	Finds the statistical median of an input column with a numeric data type. See the MEDIAN function for more information.
MAX	Returns the maximum value of the expression.
SUM	Returns the sum of the expression.
AVG	Returns the arithmetical mean of the expression.
STDDEV	Returns the standard deviation of the given expression as the square root of the VAR function.
STDDEV_POP	Returns the standard deviation of the given expression as the square root of the VAR_POP function.
STDDEV_SAMP	Returns the standard deviation of the given expression as the square root of the VAR_SAMP function.
VAR	Returns the variance of the given expression as the square of the standard deviation.
VAR_POP	Returns the population variance of expression as the sum of squares of the difference of <expression> from the mean of <expression>, divided by the number of rows remaining.
VAR_SAMP	Returns the sample variance of expression as the sum of squares of the difference of <expression> from the mean of <expression>, divided by the number of rows remaining minus 1 (one).This

Aggregate name	Description
	function is similar to VAR, the only difference is that it returns NULL when the number of rows is 1.
STRING_AGG	Returns the concatenated string.

Result type of numeric aggregate expressions

aggregate name	tinyint	smallint	integer	bigint	decimal(p,s)	decimal	real
COUNT	bigint	bigint	bigint	bigint	bigint	bigint	bigint
MIN	tinyint	smallint	integer	bigint	decimal(p,s)	decimal	real
MAX	tinyint	smallint	integer	bigint	decimal(p,s)	decimal	real
SUM	integer	integer	integer	bigint	decimal(p',s) *	decimal	real
AVG	decimal(9,6)	decimal(11,6)	decimal(16,6)	decimal(25,6)	decimal(p,s)	decimal	real
STDDEV	decimal(9,6)	decimal(11,6)	decimal(16,6)	decimal(25,6)	decimal(p,s)	decimal	real
VAR	decimal(9,6)	decimal(11,6)	decimal(16,6)	decimal(25,6)	decimal(p,s)	decimal	real

* p' is determined by the following rule: p' = 18 when p <= 18, p' = 28 when p <= 28 and p' = 38 when p <= 38

The following statements return the number of distinct combinations of column A and column B values, excluding any all-NULL tuples.

```
CREATE ROW TABLE T (A INT, B INT);
INSERT INTO T VALUES (NULL, NULL);
INSERT INTO T VALUES (1, NULL);
INSERT INTO T VALUES (1, NULL);
INSERT INTO T VALUES (NULL, 1);
INSERT INTO T VALUES (1, 1);
INSERT INTO T VALUES (1, 1);

SELECT COUNT (DISTINCT A, B) AS DISTINCT_A_B FROM T;
```

distinct_a_b
3

Based on the values in the table, there are three distinct combinations (shown in bold below) that are not an all-NULL tuple:

- A=1, B=NULL
- A=NULL, B=1
- A=1, B=1

JSON Object Expressions

The JSON object expression generates a JSON object, and looks very similar to a JSON document, including being wrapped in curly braces. A JSON object comprises one or more key-value pairs, where:

- `<key>` is similar to a string literal and must always be enclosed in double quotes.

- `<value>` is the value for the key and can be any kind of expression, including nested JSON objects enclosed in curly braces (`{ }`), or arrays enclosed in square braces (`[ ]`).

JSON object expressions are only allowed when working with JSON collection tables. They can be referenced by using statements such as SELECT, UPDATE, INSERT INTO, and can be used with operators, such as: +, -, /, *.

Syntax

```
<json_object_expression> ::= {"<key>":<json_value_expression>}
```

```
<json_value_expression> ::= '<string>'
| <numeric_literal>
| <boolean_literal>
| NULL
| <path_expression>
| <json_object_expression>
| <json_array_expression>
```

```
<json_array_expression> ::= [ <json_array_value_expression>,... ]
```

```
<json_array_value_expression> ::= '<string>'
| <numeric_literal>
| <boolean_literal>
| NULL
| <path_expression>
| <json_object_expression>
```

The following examples are valid JSON objects:

- `{ "firstname":'John', "lastname":'Smith', "age":45 }`
- `{ "firstname":'John', "lastname":'Smith', "age":45, "address": { 'street': 'Dietmar-Hopp-Allee', 'city': 'Heidelberg' } }`

Subqueries in Expressions

A subquery is a SELECT statement enclosed in parentheses. The SELECT statement can contain no more than one select list item. When used as an expression, a scalar subquery can only return a zero or a single value.

Syntax

```
<scalar_subquery_expression> ::= (<subquery>)
```

In the SELECT list of the top level SELECT, or in the SET clause of an UPDATE statement, you can use a scalar subquery anywhere where you can use a column name. scalar_subquery cannot be used inside the GROUP BY clause however.

The following statement returns the number of employees in each department for example, grouped by department name:

```
SELECT DepartmentName, COUNT(*), 'out of',
(SELECT COUNT(*) FROM Employees)
FROM Departments AS D, Employees AS E
WHERE D.DepartmentID = E.DepartmentID
GROUP BY DepartmentName;
```

Related Information

[Predicates](#)

[MEDIAN Function \(Aggregate\)](#)

[CORR_SPEARMAN Function \(Aggregate\)](#)

Predicates

A predicate is specified by combining one or more expressions, or logical operators, and returns one of the following logical/truth values: TRUE, FALSE, or UNKNOWN.

[Comparison Predicates \(ANY, SOME, and ALL\)](#)

Compares values using the specified comparison operator and returns true, false, or unknown.

[BETWEEN Predicate](#)

Compares a value with a list of values within the specified range and returns true or false.

[CONTAINS Predicate](#)

Matches a search string with the results of a subquery.

[EXISTS Predicate](#)

Tests for the presence of a value in a set and returns either true or false.

[IN Predicate](#)

Searches for a value in a set of values and returns true or false.

[LIKE Predicate](#)

Performs a comparison to see if a character string matches a specified pattern.

[LIKE REGEXPR Predicate](#)

Performs regular expression matching.

[MEMBER OF Predicate](#)

Determines whether a value is a member of an array.

[NULL Predicate](#)

Performs a comparison of the value of an expression with NULL.

Comparison Predicates (ANY, SOME, and ALL)

Compares values using the specified comparison operator and returns true, false, or unknown.

Syntax

```
<comparison_predicate> ::= <expression> { = | != | <> | > | < | >= | <= } [ ANY | SOME | ALL ] (
```

Syntax Elements

<expression_list>

```
<expression_list> ::= <expression> [{, <expression>}...]
```

An **<expression>** is either a simple expression such as a character, a date or a number, or it can be a scalar subquery.

ANY | SOME

Specifies that the comparison returns true if the comparison of the **<expression>** and at least one value returned by the **<subquery>** or **<expression_list>** is true. For example:

```
SELECT * FROM DeptTable WHERE DeptTable.LocColumn = SOME ('BOSTON','DALLAS');
```

If the `<subquery>` or `<expression_list>` is empty, the comparison returns false.

ANY and SOME, with the equal operator, is the equivalent of IN.

ALL

Specifies that the comparison returns true if the comparison of all values returned by the `<subquery>` or `<expression_list>` is true.

If the `<subquery>` or `<expression_list>` is empty, the comparison returns true. For example:

```
SELECT * FROM EmployeeTable WHERE EmployeeTable.Salary >= ALL (1400, 3000);
```

Description

Two values are compared using comparison predicates, and the comparison returns true, false, or unknown.

Related Information

[Expressions](#)

[SELECT Statement \(Data Manipulation\)](#)

BETWEEN Predicate

Compares a value with a list of values within the specified range and returns true or false.

Syntax

```
<between_predicate> ::= <expression> [NOT] BETWEEN <lower_expression> AND <upper_expression>
```

Syntax Elements

`<expression>`

The value to search for in the specified list of values.

`<lower_expression>`

An expression setting the lower bound of the value list to compare `<expression>` to.

`<upper_expression>`

An expression setting the upper bound of the value list to compare `<expression>` to.

NOT

Inverts the operation of the BETWEEN predicate: returns TRUE when `<expression>` is not in the range of values specified between `<lower_expression>` and `<upper_expression>`, including equal to `<lower_expression>` and `<upper_expression>`.

Description

The range predicate returns true if `<expressionI>` is within the range specified by `<lower_expression>` and `<upper_expression>`, and NOT is not specified.

i Note

TRUE will only be returned if *<lower_expression>* has a value less than or equal to *<upper_expression>*.

An expression is either a simple expression such as a character, a date or a number, or it can be a scalar subquery.

Related Information

[Expressions](#)

[SELECT Statement \(Data Manipulation\)](#)

CONTAINS Predicate

Matches a search string with the results of a subquery.

Syntax

```
<contains_predicate> ::= CONTAINS (
    <contains_columns>,
    <search_string>
    [, <search_specifier>] )
```

Syntax Elements

<contains_columns>

Specifies the columns to search.

```
<contains_columns> ::=
    *
    | <column_name>
    | ( <column_list> )

<column_list> ::=
    ( <column_name> [,<..>] )
```

<search_string>

Specifies the string to search for in *<contains_columns>*, using the freestyle search string format (for example, Peter "Palo Alto" OR Berlin -"SAP LABS").

```
<search_string> ::= <string_const>
```

<search_specifier>

Defines specifications on the type of matching to perform. If *<search_specifier>* is not specified, EXACT is the default.

```
<search_specifier> ::=
    [<search_type>] [<opt_search_specifier2_list>]
    | <search_specifier2_list>

<opt_search_specifier2_list> ::=
    (empty, nothing specified)
    | <search_specifier2_list>

<search_type> ::=
    <exact_search>
    | <fuzzy_search>
    | <linguistic_search>
```

```

<search_specifier2_list> ::=
  <search_specifier2>
  | <search_specifier2_list> , <search_specifier2>

<search_specifier2> ::=
  <weights>
  | <language>
  | <fulltext>

<fulltext> ::= <FULLTEXT>
  <(ON)>
  | <(OFF)>
  | (<AUTOMATIC>)

<exact_search> ::=
  EXACT
  | EXACT ( <additional_params> )

<fuzzy_search> ::=
  FUZZY
  | FUZZY ( <fuzzy_params> )
  | FUZZY ( <fuzzy_params_list> )

<fuzzy_params_list> ::= ( <fuzzy_params> ) , <fuzzy_params_list2>

<fuzzy_params_list2> ::=
  ( <fuzzy_params> )
  | <fuzzy_params_list2> , ( <fuzzy_params> )

<fuzzy_params> ::=
  <float_const>
  | <float_const> , <additional_params>
  | NULL , <additional_params>

<linguistic_search> ::=
  LINGUISTIC
  | LINGUISTIC ( <additional_params> )

<weights> ::= WEIGHT ( <float_const_list> )

<language> ::= LANGUAGE ( <string_const> )

<additional_params> ::= <string_const>

```

FUZZINESSTHRESHOLD

If the FUZZINESSTHRESHOLD parameter is defined in a join view, and *<search_specifier>* is not specified, a fuzzy search is performed using the fuzziness threshold defined in the view.

EXACT

Specifies to match the term or phrase exactly.

EXACT returns records where exact matches of the search terms are found in the search attributes.

FUZZY

Specifies to return matches that are similar to *<search_string>* (spelling errors are ignored to a certain extent for example).

Optionally, you can control the degree of fuzziness using parameters. For example, *<float_const>* specifies the degree of fuzziness expressed as value between 0.0 and 1.0, where 0.0 is very fuzzy, and 1.0 is exact. If *<float_const>* is not specified, 0.8 is the default. It is possible to override this default by defining parameter FUZZINESSTHRESHOLD supported by columnstore join views.

When FUZZY is specified with more than one *<additional_params>* parameter, the number of *<float_const>* / *<additional_params>* pairs must match the number of column names in the column list.

<additional_params>

Specifies additional search parameters in the format of key-value pairs.

WEIGHT

Specifies weightings for the columns. If a weights list is specified, it must be the same size as the number of (expanded) columns in *<contains_columns>*.

LANGUAGE

Specifies the language characteristics to apply when searching. LANGUAGE is used during preprocessing of the search string and as a pre-search filter. Only documents which match the search string and the language specified are returned.

LINGUISTIC

Specifies to perform a linguistic search. A linguistic search finds all words that have the same word stem as the search term. It also finds all words for which the search term is the word stem. For example, searching for 'cats' also returns records that contain 'cat'. You can only perform linguistic searches on columns that contain text, and for which the FAST_PREPROCESS parameter is specified as OFF.

FULLTEXT (ON | OFF | AUTOMATIC)

Specifies the behavior of the CONTAINS predicate in the presence/absence of a full text index::

ON

For columns with a full text index a full text search is done. If there is no full text index, the search fails with an error.

OFF

When OFF is specified, the CONTAINS predicate behaves as though there is no full text index, even if there is one. That is, if there is a full text index, then it is not used.

AUTOMATIC

When AUTOMATIC is specified, the CONTAINS predicate behaves according to whether there is a full text index. That is, if there is a full text index, then it is used. This is the default behavior.

Remarks

The CONTAINS predicate only works on column store objects (column tables and attribute views).

If there are multiple CONTAINS predicates specified in the WHERE clause of a SELECT statement, only one of the predicates can consist of more than one column in the *<contains_columns>* list.

There are many more search parameters (as suggested previously by *<additional_params>*) and examples for full-text search using the CONTAINS predicate found in the SAP HANA Search Developer Guide provided with SAP HANA Advanced Data Processing.

Examples

The following example performs a fuzzy search for the term cap, and returns the rows that match (Blue baseball cap, and Red car), in descending order by score:

```
CREATE SCHEMA mySchema;
CREATE COLUMN TABLE mySchema.SEARCH_TEXT( Content TEXT FAST PREPROCESS OFF, Descrip TEXT FAST PREP
INSERT INTO mySchema.SEARCH_TEXT VALUES( 'Blue baseball cap', 'Vintage', 'Out of stock');
INSERT INTO mySchema.SEARCH_TEXT VALUES( 'Red car', 'Vintage', 'Taking orders' );
INSERT INTO mySchema.SEARCH_TEXT VALUES( 'Bluish sky', 'Retro', 'Discontinued' );

SELECT SCORE() AS SCORE,*
FROM mySchema.SEARCH_TEXT
WHERE CONTAINS(CONTENT, 'cap', FUZZY(0.0))
ORDER BY SCORE DESC;
```

Changing SELECT statement to set the FUZZY parameter to 0.9, the equivalent of making the query less fuzzy or more exact, returns only the Blue baseball cap row:

```
SELECT SCORE() AS SCORE,*
FROM mySchema.SEARCH_TEXT
WHERE CONTAINS(CONTENT,'cap',FUZZY(0.9))
ORDER BY SCORE DESC
```

Using the table created in the previous example, the following statement performs an exact term search for either cap or sky, and returns the Blue baseball cap and Bluish sky rows:

```
SELECT * FROM mySchema.SEARCH_TEXT WHERE CONTAINS(CONTENT,'cap OR sky');
```

The following statement performs an exact phrase search for either baseball cap and returns the Blue baseball cap row:

```
SELECT * FROM mySchema.SEARCH_TEXT WHERE CONTAINS(CONTENT, '"baseball cap"');
```

The following statement shows a LINGUISTIC search. Searching for take returns the row that contains Taking orders because 'take' is the stem word for 'taking' in English.

```
SELECT * FROM MySchema.SEARCH_TEXT WHERE CONTAINS (*, 'take', LINGUISTIC);
```

The following statements demonstrate ways to perform a freestyle search over multiple columns. The second example is particularly helpful when you want to search across an entire table without listing the rows.

```
SELECT * FROM mySchema.SEARCH_TEXT WHERE CONTAINS (( Content, Descrip ), 'vintage');
SELECT * FROM mySchema.SEARCH_TEXT WHERE CONTAINS (*, 'vintage');
```

Related Information

[Full-Text Search with SQL](#)

EXISTS Predicate

Tests for the presence of a value in a set and returns either true or false.

Syntax

```
<exists_predicate> ::= [NOT] EXISTS ( <subquery> )
```

Syntax Elements

NOT

Inverts the operation of the EXISTS predicate: true is returned when the <subquery> returns an empty result set and false is returned when the <subquery> returns a result set.

<subquery>

Specifies what to test for. For information on subqueries, see the SELECT statement.

Description

Returns true if the *<subquery>* returns a result set that is not empty and returns false if the *<subquery>* returns an empty result set.

Related Information

[SELECT Statement \(Data Manipulation\)](#)

IN Predicate

Searches for a value in a set of values and returns true or false.

Syntax

```
<in_predicate> ::= <search_for_expression> [ NOT ] IN { <search_in_expression_list> | <subquery> }
```

Syntax Elements

<search_for_expression>

The column name or value expression to search for in the set.

<search_in_expression_list>

Specifies one or more expressions in which to search for *<search_for_expression>*.

```
<search_in_expression_list> ::= <expression> [{, <expression> }...]
```

<subquery>

Specifies a subquery to search for *<search_for_expression>* in.

NOT

Inverts the operation of the IN predicate: true is returned if *<search_for_expression>* is **not** found in the specified set.

Description

True will be returned if the value of *<search_for_expression>* is found in the *<search_in_expression_list>* or *<subquery>*.

An expression is either a simple expression such as a character, a date or a number, or it can be a scalar subquery.

Tuples are supported for *<search_for_expression>* and *<search_in_expression>*. For example, `SELECT * FROM "mySchema"."myTable" WHERE (zone, company) IN (('124110','00'), ('124116','00'));`

Tuples (list of tuples) are also allowed when using subqueries instead of *<search_in_expression>*.

While '=' (equivalent to IN) and '<>' (equivalent to NOT IN) are supported, comparisons using symbols such as '<', '>', and '!' are not supported.

Symbols '=', '<', '>' and '!=' with a list of tuples is supported only when using quantified comparison with SOME/ANY/ALL.

Examples

```
CREATE COLUMN TABLE "my_tab"
( order_nr NVARCHAR(10),
  item_nr  INTEGER,
  some_text NVARCHAR(100),
  PRIMARY KEY (order_nr, item_nr) );

INSERT INTO "my_tab" (order_nr, item_nr, some_text) VALUES ('A000000001', 1, 'A1 First Item');
INSERT INTO "my_tab" (order_nr, item_nr, some_text) VALUES ('A000000001', 2, 'A1 Second Item');
INSERT INTO "my_tab" (order_nr, item_nr, some_text) VALUES ('B000000001', 1, 'B1 First Item');
INSERT INTO "my_tab" (order_nr, item_nr, some_text) VALUES ('A000000002', 1, 'A2 First Item');
INSERT INTO "my_tab" (order_nr, item_nr, some_text) VALUES ('A000000002', 2, 'A2 Second Item');

SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) = ('A000000001', 2);

SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) <> ('A000000001', 2);

SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) != ('A000000001', 2);

-- works only when sub-select returns zero or one row
SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) != (select order_nr, item_nr from "my_tab" where some_text like 'B1%

-- error: feature not supported: only '=' and '<>/'!=' operators are allowed here
SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) > ('A000000001', 2);

-- quantified comparison with SOME/ANY/ALL and sub-select is also supported
SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) <> ALL (select order_nr, item_nr from "my_tab" where some_text like 'B

-- IN predicate is equivalent to quantified comparison = ANY/SOME
SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) in (('A000000001', 2), ('B000000001', 1));
SELECT * FROM "my_tab"
  WHERE (order_nr, item_nr) = ANY (('A000000001', 2), ('B000000001', 1));
```

Related Information

[SELECT Statement \(Data Manipulation\)](#)
[Expressions](#)

LIKE Predicate

Performs a comparison to see if a character string matches a specified pattern.

Syntax

```
<like_predicate> ::= <source_expression> [ NOT ] LIKE <pattern_expression>
[ ESCAPE <escape_expression> ]
```

Syntax Elements

<source_expression>

Specifies the column or character string in which to search for *<pattern_expression>*. Specifying NOT inverts the operation of the LIKE predicate.

<pattern_expression>

Specifies the pattern to search for in *<source_expression>*.

<escape_expression>

Specifies the escape character used in comparison string *<pattern_expression>*, if any.

Description

The LIKE predicate performs string comparisons: *<source_expression>* is tested for the pattern contained in *<pattern_expression>*. LIKE returns true if the value of *<pattern_expression>* is found in *<source_expression>* (assuming NOT is not set).

Wildcard characters (%) and (_) can be used in *<pattern_expression>*. The percentage sign (%) wildcard matches zero or more characters. The underscore (_) wildcard matches exactly one character.

To match a percentage sign or underscore with the LIKE predicate, an escape character must be placed in front of the wildcard character. Use ESCAPE *<escape_expression>* to specify the escape character you are using. For example, LIKE 'data_%' ESCAPE '_' matches the string data%, and LIKE 'data__%' ESCAPE '_' (that is, two underscores, followed by a percent sign) matches a string that starts with 'data_'

The underscore (_) and percentage sign (%) are ASCII characters.

An expression is either a simple expression such as a character, a date or a number, or it can be a scalar subquery.

If any of *<source_expression>*, *<pattern_expression>*, or *<escape_expression>* is NULL, then the predicate returns UNKNOWN.

Related Information

[Expressions](#)

[Expressions](#)

[SELECT Statement \(Data Manipulation\)](#)

LIKE_REGEXPR Predicate

Performs regular expression matching.

Syntax

```
<regex_subject_string> LIKE_REGEXPR <pattern> [ FLAG <flag> ]
```

Syntax Elements

<regex_subject_string>

Specifies the string that the search pattern should be applied to. If *<regex_subject_string>* is empty, then the result is empty.

```
<regex_subject_string> ::= <string>
```

<pattern>

Specifies a search pattern based on Perl Compatible Regular Expression (PCRE).

```
<pattern> ::= !!Perl Compatible Regular Expression
```

Specifies the matching behavior of the predicate.

<flag> ::= i | m | s | x | U

Flag options

Flag option	Description
i	Enables case-insensitive matching
m	Enables multiline mode, where the <subject_string> will be treated as multiple lines and the expression ^ and \$ match just after or just before, respectively, a line terminator or the end of the input sequence
s	Enables the expression <.> as a wildcard to match any character, including a line terminator
x	Permits whitespace and comments in the pattern
U	SAP HANA uses the Perl-compatible Regular Expressions (PCRE) library to process regular expressions. Specifying 'U' (short for "ungreedy") inverts the "greediness" of quantifiers so that they are not greedy by default but become greedy only when followed by "?". Ungreedy matching can often perform faster because it finds the shorter match at times when it is only interesting to know whether there is any match. For a full understanding of the "greedy" versus "ungreedy" matching behavior of the Perl-compatible Regular Expressions (PCRE) library, visit:https://www.pcre.org/original/doc/html/pcrematching.html ➦ .

Description

This predicate performs regular expression matching.

If any of <pattern>, <flag>, or <regex_subject_string> is NULL, then the predicate returns UNKNOWN.

Example

This fictitious example searches for text like them or this in the table mytab:

```
SELECT * FROM mytab WHERE text LIKE_REGEXPR ' them|this ';
```

MEMBER OF Predicate

Determines whether a value is a member of an array.

Syntax

<expression> [NOT] MEMBER OF <array_value_expression>

Description

Determines whether a value is a member of an array.

Example

The following statements create a table and insert arrays into it:

```
CREATE COLUMN TABLE ARRAY_TEST (IDX INT, VAL INT ARRAY);
INSERT INTO ARRAY_TEST VALUES (1, ARRAY(1, 2, 3));
INSERT INTO ARRAY_TEST VALUES (2, ARRAY(10, 20, 30, 40));
INSERT INTO ARRAY_TEST VALUES (3, ARRAY(10, 20, 30, 40));
INSERT INTO ARRAY_TEST VALUES (4, ARRAY(80, 90, 100, 110));
```

The following statement tests for arrays where 10 is a member, and returns the IDX values for the qualifying arrays.

```
SELECT IDX FROM ARRAY_TEST WHERE 10 MEMBER OF VAL;
```

IDX
2
3

The following statement tests for arrays where 10 is not a member, and returns the IDX values for the qualifying arrays.

```
SELECT IDX FROM ARRAY_TEST WHERE 10 NOT MEMBER OF VAL;
```

IDX
1
4

NULL Predicate

Performs a comparison of the value of an expression with NULL.

Syntax

```
<null_predicate> ::= <expression> IS [NOT] NULL
```

Syntax Elements

<expression>

An expression is either a simple expression such as a character, a date or a number, or it can be a scalar subquery.

IS NULL

Returns true if the value of <expression> is NULL.

IS NOT NULL

Returns true if the value of <expression> is not NULL.

Description

Performs a comparison of the value of an expression with NULL.

SELECT Statement (Data Manipulation)

Session Variables

The following table lists predefined session variables and whether they can be set by the user (using a SET statement) or a client (API) call, or whether they are set exclusively by the server.

Predefined Variable (M_SESSION_CONTEXT.KEY)	Value Constraint	Set by	In M_SESSION_CONTEXT	Description
APPLICATION	NVARCHAR(256)	User/Client	Yes	Specifies the application name. Server usage: <code>@@APPLICATION</code>
APPLICATIONVERSION	NVARCHAR(256)	User/Client	Yes	Specifies the application version. Application version format is <code>major.minor.revision</code> . Server usage: <code>@@APPLICATIONVERSION</code>
APPLICATIONUSER	NVARCHAR(256)	User/Client	Yes	Specifies the application user. Server usage: <code>@@APPLICATIONUSER</code>
APPLICATIONACTION	NVARCHAR(256)	User/Client	Yes	Specifies the application action. The application action is the name of the application currently processing the request, and could be as coarse as the application name or as fine as the request URI. The application action is a string of up to 256 characters. Server usage: <code>@@APPLICATIONACTION</code>
APPLICATIONCOMPONENT	NVARCHAR(64)	Client	Yes	Specifies the application component. Server usage: <code>@@APPLICATIONCOMPONENT</code>
APPLICATIONCOMPONENTTYPE	VARCHAR(32)	Client	Yes	Specifies the application component type. The application component type is a string of up to 32 characters. For example, <code>U</code> for user, <code>C</code> for component, and <code>S</code> for session. Server usage: <code>@@APPLICATIONCOMPONENTTYPE</code>
APPLICATIONSOURCE	NVARCHAR(256)	User/Client	Yes	Specifies the application source. The application source is the name of the application that called the application. The application source is a string of up to 256 characters. The usage is as follows: <code><application></code> <code><application>.<component></code> <code><application>.<component>.<session></code> Server usage: <code>@@APPLICATIONSOURCE</code>
CASE_SENSITIVE	VARCHAR(5)	User/Client	Only if set to FALSE	Specifies the case sensitivity of the application. The exact meaning of the value is not defined.

Predefined Variable (M_SESSION_CONTEXT.KEY)	Value Constraint	Set by	In M_SESSION_CONTEXT	Description
				FALSE, any string is fou If a case-ins of a filter or returned in function is result to ac SELECT * works well, LIKE '%C WHERE na uppercase Also, CASE, NVARCHAF operations: • WH • JOI • HA\ • CAS • GRO • ORI • Wir • UNI Server usa search.
DATE_FORMAT	-	User/Client	Yes	Specifies th session. Server usa
DEBUG_TOKEN	VARCHAR(32)	User/Client	Yes	Specifies th session var criterion. If you speci connection attaches or session var Server usa M_DEBUG_
ENABLE_ABSTRACT_SQL_PLAN_ENTRY		User/Client	Yes	Specifies th current ses When the q appended t the plan ca

Predefined Variable (M_SESSION_CONTEXT.KEY)	Value Constraint	Set by	In M_SESSION_CONTEXT	Description
PROTOCOL_VERSION	-	Server	Yes	Specifies the libraries, for (<i><distributed></i> <i><data_formats></i> Server usage
SECONDDATE_FORMAT		User/Client	Yes	Specifies the session Server usage
SPATIAL_OUTPUT_REPRESENTATION	-	User/Client	Yes	Specifies the ST_Point. V
TEMPORAL_SYSTEM_TIME_AS_OF	-	User/Client	Yes	For use with session var SYSTEM_TIME tables cont (assuming specified in Server usage
TEMPORAL_APPLICATION_TIME_AS_OF	-	User/Client	Yes	For use with session var APPLICATION time period statements OF is not al Server usage
TIME_FORMAT	-	User/Client	Yes	Specifies the session. Server usage
TIMESTAMP_FORMAT	-	User/Client	Yes	Specifies the session Server usage
TOTAL_ROWCOUNT	Integer	Server	No, only accessible from: SELECT session_context('TOTAL_ROWCOUNT') FROM DUMMY.	Specifies the LIMIT x return ROWCOUNT in TOTAL_ROW deleted/res The similar extension. This feature column view count or on depends or internally c recommend show infor 50000".

Predefined Variable (M_SESSION_CONTEXT.KEY)	Value Constraint	Set by	In M_SESSION_CONTEXT	Description
				Server user
TRACEPROFILE	-	User/Client	Yes	Specifies the manual act [traceprofil The value c comma sep In SAP HANA End-To-End Server user

Caution

Session variables set by User can result in unexpected behavior in cases of scale-out system. For more information, see [2215929](#) .

Related Information

[M_SESSION_CONTEXT System View](#)

[SET \[SESSION\] Statement \(Session Management\)](#)

[UNSET \[SESSION\] Statement \(Session Management\)](#)

SQL Functions

Documents the built-in SQL functions that are provided with SAP HANA.

Analytic functions are positioned in this guide as aggregate functions and window aggregate functions.

[Alphabetical List Of Functions](#)

[Aggregate Functions](#)

Aggregate functions are analytic functions that calculate an aggregate value based on a group of rows.

[Array Functions](#)

Array functions take arrays as input.

[Data Type Conversion Functions](#)

Data type conversion functions convert data from one data type to another data type.

[Datetime Functions](#)

Date and time functions perform operations on date and time data types or return date or time information.

[Fulltext Functions](#)

Fulltext functions perform operations on data that has a fulltext index.

[Hierarchy Functions](#)

Hierarchy functions allow you to work with hierarchical data such as tables with rows arranged in a tree or directed graph.

[JSON Functions](#)

JSON functions are functions that return or operate on JSON data.

[Miscellaneous Functions](#)

SAP HANA supports many functions that return system values and perform various operations on values, expressions, and return values of other functions.

[Numeric Functions](#)

Numeric functions perform mathematical operations on numerical data types or return numeric information.

[Security Functions](#)

Security functions provide special functionality for security purposes.

[Series Data Functions](#)

Series data functions provide special functionality for series data and series tables.

[String Functions](#)

String functions perform extraction and manipulation on strings, or return information about strings.

[Window Functions and the Window Specification](#)

Window functions allow you to perform analytic operations over a set of input rows.

Aggregate Functions

Aggregate functions are analytic functions that calculate an aggregate value based on a group of rows.

Supported functions:

- [AUTO_CORR Function \(Aggregate\)](#)
- [AVG Function \(Aggregate\)](#)
- [CORR Function \(Aggregate\)](#)
- [CORR_SPEARMAN Function \(Aggregate\)](#)
- [COUNT Function \(Aggregate\)](#)
- [CROSS_CORR Function \(Aggregate\)](#)
- [DFT Function \(Aggregate\)](#)
- [FIRST_VALUE Function \(Aggregate\)](#)
- [LAST_VALUE Function \(Aggregate\)](#)
- [MAX Function \(Aggregate\)](#)
- [MEDIAN Function \(Aggregate\)](#)
- [MIN Function \(Aggregate\)](#)
- [NTH_VALUE Function \(Aggregate\)](#)
- [STDDEV Function \(Aggregate\)](#)
- [STDDEV_POP Function \(Aggregate\)](#)
- [STDDEV_SAMP Function \(Aggregate\)](#)
- [STRING_AGG Function \(Aggregate\)](#)
- [SUM Function \(Aggregate\)](#)
- [VAR Function \(Aggregate\)](#)
- [VAR_POP Function \(Aggregate\)](#)
- [VAR_SAMP Function \(Aggregate\)](#)

Related Information

[Window Aggregate Functions](#)

Array Functions

Array functions take arrays as input.

Supported functions:

- [CARDINALITY Function \(Array\)](#)
- [MEMBER_AT Function \(Array\)](#)
- [SUBARRAY Function \(Array\)](#)
- [TRIM_ARRAY Function \(Array\)](#)

Data Type Conversion Functions

Data type conversion functions convert data from one data type to another data type.

Data type conversion functions are used to convert function arguments from one data type to another, or to test whether a conversion is possible.

i Note

In both implicit and explicit numeric type conversions, these functions always truncate the least significant digits toward zero.

Supported functions:

- [CAST Function \(Data Type Conversion\)](#)
- [TO_ALPHANUM Function \(Data Type Conversion\)](#)
- [TO_BIGINT Function \(Data Type Conversion\)](#)
- [TO_BINARY Function \(Data Type Conversion\)](#)
- [TO_BLOB Function \(Data Type Conversion\)](#)
- [TO_BOOLEAN Function \(Data Type Conversion\)](#)
- [TO_CLOB Function \(Data Type Conversion\)](#)
- [TO_DATE Function \(Data Type Conversion\)](#)
- [TO_DATS Function \(Data Type Conversion\)](#)
- [TO_DECIMAL Function \(Data Type Conversion\)](#)
- [TO_DOUBLE Function \(Data Type Conversion\)](#)
- [TO_FIXEDCHAR Function \(Data Type Conversion\)](#)
- [TO_INT Function \(Data Type Conversion\)](#)

- [TO_INTEGER Function \(Data Type Conversion\)](#)
- [TO_JSON_BOOLEAN \(Data Type Conversion\)](#)
- [TO_NCLOB Function \(Data Type Conversion\)](#)
- [TO_NVARCHAR Function \(Data Type Conversion\)](#)
- [TO_REAL Function \(Data Type Conversion\)](#)
- [TO_SECONDDATE Function \(Data Type Conversion\)](#)
- [TO_SMALLDECIMAL Function \(Data Type Conversion\)](#)
- [TO_SMALLINT Function \(Data Type Conversion\)](#)
- [TO_TIME Function \(Data Type Conversion\)](#)
- [TO_TIMESTAMP Function \(Data Type Conversion\)](#)
- [TO_TINYINT Function \(Data Type Conversion\)](#)
- [TO_VARCHAR Function \(Data Type Conversion\)](#)

Datetime Functions

Date and time functions perform operations on date and time data types or return date or time information.

Supported functions:

- [ADD_DAYS Function \(Datetime\)](#)
- [ADD_MONTHS Function \(Datetime\)](#)
- [ADD_MONTHS_LAST Function \(Datetime\)](#)
- [ADD_NANO100 Function \(Datetime\)](#)
- [ADD_SECONDS Function \(Datetime\)](#)
- [ADD_WORKDAYS Function \(Datetime\)](#)
- [ADD_YEARS Function \(Datetime\)](#)
- [CURRENT_DATE Function \(Datetime\)](#)
- [CURRENT_MVCC_SNAPSHOT_TIMESTAMP Function \(Datetime\)](#)
- [CURRENT_TIME Function \(Datetime\)](#)
- [CURRENT_TIMESTAMP Function \(Datetime\)](#)
- [CURRENT_UTCDATE Function \(Datetime\)](#)
- [CURRENT_UTCTIME Function \(Datetime\)](#)
- [CURRENT_UTCTIMESTAMP Function \(Datetime\)](#)
- [DAYNAME Function \(Datetime\)](#)
- [DAYOFMONTH Function \(Datetime\)](#)

- [DAYOFYEAR Function \(Datetime\)](#)
- [DAYS_BETWEEN Function \(Datetime\)](#)
- [EXTRACT Function \(Datetime\)](#)
- [HOUR Function \(Datetime\)](#)
- [ISOWEEK Function \(Datetime\)](#)
- [LAST_DAY Function \(Datetime\)](#)
- [LOCALTOUTC Function \(Datetime\)](#)
- [MINUTE Function \(Datetime\)](#)
- [MONTH Function \(Datetime\)](#)
- [MONTHNAME Function \(Datetime\)](#)
- [MONTHS_BETWEEN Function \(Datetime\)](#)
- [NANO100_BETWEEN Function \(Datetime\)](#)
- [NEXT_DAY Function \(Datetime\)](#)
- [NOW Function \(Datetime\)](#)
- [QUARTER Function \(Datetime\)](#)
- [SECOND Function \(Datetime\)](#)
- [SECONDS_BETWEEN Function \(Datetime\)](#)
- [UTCTOLOCAL Function \(Datetime\)](#)
- [WEEK Function \(Datetime\)](#)
- [WEEKDAY Function \(Datetime\)](#)
- [WORKDAYS_BETWEEN Function \(Datetime\)](#)
- [YEAR Function \(Datetime\)](#)
- [YEARS_BETWEEN Function \(Datetime\)](#)

Fulltext Functions

Fulltext functions perform operations on data that has a fulltext index.

Supported functions:

- [GROUP_SCORE Function \(Miscellaneous\)](#)
- [INDEXING_ERROR_CODE Function \(Fulltext\)](#)
- [INDEXING_ERROR_MESSAGE Function \(Fulltext\)](#)
- [INDEXING_STATUS Function \(Fulltext\)](#)
- [LANGUAGE Function \(Fulltext\)](#)